

MEMORIA TÉCNICA

1. Problema y Oportunidad

La congestión vehicular en áreas comerciales de alta densidad en Mendoza (como el microcentro) genera ineficiencia urbana debido a que los conductores carecen de información previa sobre la disponibilidad de estacionamiento. Esto causa recorridos innecesarios, aumento de emisiones de CO₂ y fomenta un mercado informal de cobro sin trazabilidad fiscal ni control institucional.

Usuarios afectados

- Conductores urbanos que pierden tiempo buscando lugar sin información en tiempo real.
- Municipios que no tienen visibilidad sobre la ocupación real del espacio público ni pueden auditar la recaudación de forma transparente.
- Comercios y Ciudadanos afectados por la congestión vial y el impacto ambiental derivado.

Magnitud y relevancia

Un estudio de INRIX (2022) estimó que el conductor promedio en ciudades latinoamericanas dedica entre 40 y 60 horas anuales a buscar estacionamiento. Extrapolado a Mendoza Capital (con aprox. 120.000 vehículos activos), esto representa más de 6 millones de horas/año de tiempo productivo perdido, y un volumen de emisiones innecesarias equivalente a toneladas de CO₂ evitables.

La validación del problema se realizó a través de observación directa del comportamiento en zonas de alta densidad vehicular del centro mendocino.

2. Solución Propuesta

¿Qué es SmartPark?

SmartPark es un sistema de gestión inteligente de estacionamiento urbano diseñado para municipios y entidades de tránsito (modelo B2G — Business to Government). Combina tres tecnologías en una plataforma integrada:

- Computer Vision (OpenCV + MOG2) para detectar la presencia de vehículos en tiempo real.
- Blockchain (Ethereum Sepolia + Solidity) para registrar cada evento de manera inmutable y auditable.
- Aplicación web en tiempo real (Next.js + Supabase) para que el ciudadano reserve su lugar y el municipio visualice el estado del estacionamiento.

Funcionamiento general

La solución opera en tres capas coordinadas. Primero, una cámara conectada a la maqueta física detecta mediante sustracción de fondo (algoritmo MOG2) si un espacio está

libre u ocupado. Esta detección viaja al backend (Flask) que actualiza la base de datos en Supabase y refleja el cambio en el mapa del frontend en menos de 3 segundos. Simultáneamente, el backend sella el evento en la blockchain de Ethereum Sepolia a través del contrato inteligente ParkingRegistry, generando un registro permanente e inalterable con patente, espacio, timestamp y monto pagado.

Diferenciación respecto a alternativas

Característica	SmartPark	Apps tradicionales	Parquímetros físicos
Tiempo real CV	Sí	Parcial	No
Registro blockchain	Sí	No	No
Recaudación auditable	Sí	Limitado	Opaco
Tokenización ARS→stablecoin	Sí	No	No
Adaptado al contexto ARG	Sí (MercadoPago)	Parcial	Sí

Criterio de Elección del Usuario

El conductor elige la app porque elimina la frustración de buscar lugar mediante un mapa interactivo en tiempo real y permite pagar la tasa digitalmente sin efectivo ni intermediarios informales. **El municipio** elige el sistema porque reduce costos de infraestructura (una sola cámara vigila múltiples boxes mediante software, evitando sensores físicos por plaza) y garantiza transparencia fiscal inmutable frente a auditorías.

3. MVP Desarrollado

El MVP de SmartPark es un sistema funcional de punta a punta que cubre el ciclo completo: detección visual → reserva → pago → registro blockchain → visualización en mapa. A continuación se detalla el alcance real logrado:

- **Frontend (Next.js, puerto 3000):** Mapa interactivo con lugares de estacionamiento en tiempo real, coloreados según estado. El mapa realiza polling al backend cada 3 segundos para reflejar cambios automáticamente. Incluye modal de reserva con simulación de pago MercadoPago e historial de reservas.
- **Backend (Flask, puerto 8000):** API REST con 4 endpoints operativos (GET /api/spaces, POST /api/reserve, POST /api/occupancy, GET /api/spaces/<id>/history). Gestiona la lógica de negocio, escribe en Supabase y sella en blockchain.
- **Computer Vision (cv_detector.py):** Modo demo (teclado) y modo producción (cámara + MOG2). Define ROIs por calibración, detecta ocupación y notifica al backend.
- **Blockchain (Ethereum Sepolia):** 4 contratos desplegados y verificados: ParkingRegistry (auditoría), SparkToken (tokenización), MockUSDC (stablecoin simulada) y SimpleSwap (pool de swap SPARK↔mUSDC).

- **Base de datos (Supabase/PostgreSQL):** 3 tablas: parking_spaces, reservations, occupancy_events. Schema tipado con restricciones de integridad referencial.

Estados del sistema

Color en mapa	Estado	Significado
Verde	available	Libre, disponible para reservar
Azul	reserved	Reserva pagada, el auto aún no llegó
Violeta	occupied_valid	Auto detectado por CV con reserva válida
Rojo	occupied_illegal	Auto detectado por CV sin reserva (intruso)

Funcionalidades pendientes para próximas versiones: OCR real de patentes, pago real con MercadoPago API, notificaciones push al usuario cuando el lugar reservado es detectado como ocupado por otro vehículo, dashboard de analítica para el municipio (métricas de ocupación, ingresos, patrones horarios).

4. Componentes Tecnológicos - Stack tecnológico

Capa	Tecnología	Rol en el sistema
Frontend	Next.js 14 + TypeScript + Tailwind	UI del usuario: mapa, reservas, historial
Mapas	Leaflet.js	Visualización de pins por estado en tiempo real
Backend API	Python Flask	Lógica de negocio, orquestación CV + DB + BC
Base de datos	Supabase (PostgreSQL)	Estado vivo del sistema, reservas y eventos
Computer Vision	OpenCV (MOG2 BGS)	Detección de ocupación por diferencia de fondo
Blockchain	Solidity + Ethereum Sepolia	Registro inmutable de reservas y eventos
Web3 client	web3.py	Interacción backend→blockchain
Pagos (mock)	MercadoPago (simulado)	Flujo de pago familiar para el usuario argentino

Inteligencia Artificial - Computer Vision

Para eliminar la necesidad de instalar costosos sensores físicos en el asfalto, SmartPark utiliza una sola cámara común controlada por el script cv_detector.py. Este módulo procesa el video en tiempo real mediante OpenCV y aplica el algoritmo estadístico MOG2 (Sustracción de Fondo). MOG2 analiza cada píxel de la imagen y se adapta automáticamente a los cambios de luz naturales del entorno (como las sombras o el polvo) sin saturar el procesador de la computadora.

El Pipeline (proceso) de detección funciona en 4 pasos rápidos:

1. **Captura:** Toma el cuadro de la cámara.
2. **Filtrado:** Aplica el algoritmo MOG2 para aislar el movimiento del fondo.
3. **Enfoque (ROI):** Analiza exclusivamente las Regiones de Interés (los 4 boxes de estacionamiento delimitados).
4. **Notificación:** Cuenta los píxeles modificados; si superan un umbral asignado, el espacio se marca como ocupado y envía un aviso inmediato al backend mediante un POST /api/occupancy.

Antes de subir cualquier dato a la blockchain, el backend actúa como un **analizador léxico** utilizando expresiones regulares. Este sistema funciona como un filtro inteligente que comprueba que el texto de la patente cumpla estrictamente con la sintaxis oficial argentina (el formato viejo AAA000 o el nuevo del Mercosur AA000AA). Si la lectura de la cámara arroja un texto malformado o erróneo, el sistema lo rechaza en el origen (Error 400), evitando gastar recursos en la red de bloques y garantizando la integridad absoluta de la base de datos.

Blockchain - Contratos Inteligentes

Red utilizada: Ethereum Sepolia (testnet pública). Todos los contratos están desplegados y verificables en Etherscan.

Contrato	Dirección (Sepolia)	Función
ParkingRegistry	0xAA1D0Db2...b44AF	Registro inmutable de reservas y eventos de ocupación
SparkToken	0x06d3879d...d808	ERC-20 que representa pesos recaudados (1 ARS = 1 SPARK)
MockUSDC	0xaC9063...AE23a	Stablecoin simulada para el swap
SimpleSwap	0xb8a0f8...810c	Pool AMM (constant product $x*y=k$) SPARK↔mUSDC

Control de acceso: solo wallets autorizadas (authorizedReporters) pueden registrar eventos en ParkingRegistry. Esto impide que actores externos inserten eventos falsos y garantiza la integridad del log de recaudación. La wallet del backend es la única autorizada en el MVP.

Integración con contenidos de la materia

Unidades I y III (Compiladores): El backend actúa como un **Autómata Finito Determinista (Scanner)** mediante expresiones regulares para validar la sintaxis de las patentes argentinas, rechazando en el origen cadenas inválidas (Error 400).

Unidad II (Sistemas de Tipos y Abstracción): Encapsulamiento de la lógica criptográfica en Solidity e implementación de un **esquema rígido de datos** mediante restricciones ENUM en base de datos y tipado estático en TypeScript para evitar mutaciones inválidas.

Unidad IV (Seguridad y Blockchain): Control de acceso mediante **criptografía** asimétrica, exigiendo que cada evento sea firmado digitalmente por la wallet del backend (authorizedReporters) para crear un historial inmutable y auditable de tipo *append-only*.

5. Innovación e Impacto

- Web3 como motor de auditoría pública
- Tokenización anti-inflacionaria
- Arquitectura de capas desacopladas: CV, backend, base de datos y blockchain funcionan de forma independiente.
- UX familiar para el usuario argentino: El ciudadano no necesita conocer qué es blockchain ni tener una wallet. La Web3 opera completamente en el backend.

Impacto social: reducción del tiempo que el conductor dedica a buscar estacionamiento. Eliminación del mercado informal de cobro sin trazabilidad. Formalización de trabajadores informales del sector (posibilidad de integrarlos como operadores del sistema).

Impacto ambiental: si SmartPark reduce 5 minutos por evento de estacionamiento en un área con 500 eventos/día, el ahorro diario es de 2.500 minutos de motor encendido en circulación innecesaria.

Impacto económico: el municipio obtiene datos de ocupación en tiempo real para fijar tarifas dinámicas, detectar zonas subutilizadas y demostrar la transparencia de la recaudación a organismos de control. Modelo de negocio: SaaS con abono mensual al municipio (software + soporte + actualización de algoritmos).

Métricas de éxito propuestas

Tasa de detección correcta del CV > 95% en condiciones controladas de la maqueta, latencia de actualización del mapa ≤ 3 segundos desde la detección, reservas registradas en blockchain, reducción medible del tiempo de búsqueda de estacionamiento en zona piloto.

6. Viabilidad y Sustentabilidad

Factibilidad técnica

El stack tecnológico es 100% open source (OpenCV, Flask, Next.js, Supabase free tier, Solidity). El prototipo está funcionando en hardware convencional (laptop + webcam). Para producción, se requiere una Raspberry Pi 4 por zona de cámaras y un servidor en la nube para el backend (AWS/GCP). La infraestructura blockchain es pública y no requiere mantenimiento propio.

Factibilidad económica

Modelo SaaS B2G: el municipio paga un abono mensual que cubre infraestructura cloud, soporte técnico y actualizaciones de algoritmos. El costo de hardware por zona (cámara IP

+ mini-PC) es de aproximadamente USD 150, amortizable en 2 años con la recaudación del espacio. Las transacciones en Sepolia son gratuitas; en mainnet, el costo de gas por evento es despreciable comparado con el valor del registro.

Posibles aliados y estrategia de crecimiento

- Municipio de Mendoza / cliente B2G inicial.
- Dirección de Tránsito provincial: acceso a cámaras de tránsito existentes.
- Universidad Champagnat: aliado académico para iteración del algoritmo CV.
- MercadoPago: integración real de la pasarela de pago en v2.0.

Estrategia de crecimiento: comenzar con un piloto en 10 espacios de una zona de alto tráfico, medir ROI y replicar por zona.

7. Trabajo en Equipo

Roles y distribución de tareas

Integrante	Rol	Responsabilidades principales
Joaquín	Lead Frontend & Maker	Dashboard Next.js, mapa Leaflet, maqueta física 3D, integración frontend-backend
Nicolas	IA & Python Dev	Script cv_detector.py, modos demo/calibración/producción, integración con API
Nicolas, Joaquin	Web3 & Backend Engineer	Contratos Solidity, despliegue en Sepolia, Flask API, schema Supabase
Luciana	Documentación, Pitch, Lean Canvas	Memoria técnica, Lean Canvas, Pitch Deck, video demo, README del repositorio

Organización y metodología

El equipo adoptó un enfoque ágil basado en el backlog oficial, priorizando los requerimientos mediante el criterio **MoSCoW**. La ejecución se dividió de forma estratégica según las especialidades de cada integrante. La sincronización se consolidó mediante reuniones presenciales en la universidad y canales de comunicación rápida para el seguimiento diario a la par.

8. Aprendizajes y Propuestas Descartadas

Ideas evaluadas y descartadas

- **OCR de patentes en tiempo real (Tesseract) y pago real con MercadoPago API**

Principales aprendizajes

- La integración de múltiples tecnologías (CV + Flask + Supabase + web3.py + Next.js) requiere contratos de API bien definidos desde el inicio. El diseño del endpoint POST /api/occupancy como único punto de entrada para el detector CV simplificó enormemente la coordinación.
- Ethereum Sepolia como testnet pública es ideal para prototipado: gratuita, con herramientas maduras (Hardhat, Etherscan, faucets) y comportamiento equivalente a mainnet.
- El manejo de errores asíncronos en la blockchain (falta de gas, timeout de RPC) debe estar pensado desde el diseño: la decisión de hacer el registro blockchain no-bloqueante fue clave para la robustez del MVP.

9. Fuentes y Referencias

- INRIX Global Traffic Scorecard 2022 — inrix.com/scorecard/
- OpenCV
- Ethereum Sepolia Testnet
- Supabase Documentation
- Solidity Language Reference v0.8.x